

LLM APIs

Recap

Context window, KV cache, token cost, API parameters — now that you know the Transformer, these land properly

AI-LM-IN-TH-000001-6 · M3 Recap

25 min

Duration

B3 – Apply

Bloom's

L2 – Foundational

Level

Topics 1–4

Unlocked by

Learning Objectives

By the end of this session, you will be able to:

B2 – Define

The context window precisely: total tokens the model can attend to in one forward pass

B2 – Explain

What the KV cache stores, why it exists, and what it costs in GPU memory

B3 – Distinguish

TTFT vs TPOT, and explain why prefill is parallel but decoding is sequential

B3 – Calculate

Token cost for a real API call: system prompt + history + context + output $\times$ per-token price

B3 – Map

Each API parameter (temperature, top_p, max_tokens, stop, stream, seed) to its underlying mechanism

B4 – Evaluate

The latency–throughput–cost tradeoff and apply cost-reduction strategies

Session Roadmap

5 sub-topics — deferred from Topics 1–3, now fully grounded

5.1 The Context Window — What It Actually Is

Not a memory system

5.2 Tokens Are Currency — Pricing and Limits

Input, output, and cost arithmetic

5.3 The KV Cache — Why It Exists and What It Costs

Connects to Topic 3 self-attention

5.4 API Parameters Map to Generation Concepts

*temperature · top_p · stream · seed
· ...*

5.5 Latency, Throughput, and the Cost Tradeoff Triangle

TTFT · TPOT · production decisions

Why deferred? Context window and KV cache require Topic 3 (full Transformer block) to land precisely. Positional encoding derivation → M2.4. Flash Attention / vLLM → M9.

5.1

The Context Window — What It Actually Is

Not a memory system. Not a database. A fixed-length sequence entering the Transformer.

The Context Window — Precise Definition

Everything the model can attend to in a single forward pass

The context window is the total number of tokens the model can attend to in one forward pass — both input (prompt) and output (generated response) combined. It is the fixed-length sequence that enters the Transformer.

What the context window IS

- The flat sequence of token IDs entering the Transformer
- Attended to by every other token via self-attention (within causal mask)
- Both prompt and generated tokens occupy the same window
- Once full, the oldest tokens are simply dropped — no gradual fade
- A hard architectural limit, not a soft preference

What the context window is NOT

- A persistent memory system (tokens outside = zero access)
- A database the model queries
- A store of long-term knowledge about the user
- A buffer with partial memory of older content
- Unlimited or expandable at inference time

Topic 3 connection: the context window limit is a positional encoding design choice, not just a memory constraint. Learned absolute PE hard-limits to training length. RoPE with LongRoPE scaling is why Llama 3.1 can extend from 8K to 128K post-hoc without retraining.

Production Context Windows — What the Numbers Mean

Anchoring the abstract token count in tangible real-world units

GPT-4o (OpenAI)

128K

~256 pages of text

128K

Claude 3.5 (Anthropic)

200K

~400 pages / a full novel

200K

Llama 3.1 (Meta, open)

128K

~256 pages of text

128K

Gemini 1.5 Pro (Google)

1M

~2000 pages / hour of video

1M

Rule of thumb: 1 page of English text $\approx$ 500 tokens. 1 line of Python $\approx$ 10-15 tokens. 1 minute of speech transcript $\approx$ 150 tokens.

5.2

Tokens Are Currency — Pricing and Limits

Every API call is billed in tokens. Input and output are priced differently.

Anatomy of a Token Bill

Every token in the context window costs money. Where they come from matters.

Input token components — all concatenated into one sequence before the first forward pass:

~500

~1500

~2000

System Prompt

~500 tokens

Prepended at position 0. Attended by every subsequent token. Costs money on every call.

Conversation History

~1500 tokens

Grows with every turn. The primary driver of context exhaustion in long dialogues.

Retrieved Context (RAG)

~2000 tokens

Largest cost driver in RAG pipelines. Each retrieved chunk adds to the bill.

User Query

~50 tokens

Usually the smallest component. Rarely the bottleneck.

Output tokens (~300) are billed separately and cost 3–5× more per token than input. Each output token requires one full autoregressive forward pass.

Worked Billing Example — A Typical RAG Query

Real arithmetic. Build this reflex for every model call you write.

Scenario: RAG-powered Q&A system using GPT-4o | Pricing: \$2.50 / 1M input tokens · \$10.00 / 1M output tokens

Component	Tokens	Category	Cost at GPT-4o rates
System prompt	500	Input	\$0.00125
Conversation history (3 turns)	900	Input	\$0.00225
Retrieved chunks (3 × ~600 tok)	1 800	Input	\$0.00450
User query	50	Input	\$0.000125
Total input	3 250	Input	\$0.008125
Generated response	300	Output	\$0.003000
Total per query	3 550	—	\$0.011125

At 10 000 queries / day: \$111.25 / day → \$3 337.50 / month → \$40 056 / year. System prompt caching can cut 90% of that cost on the cached prefix.

5.3

The KV Cache — Why It Exists and What It Costs

Self-attention computes K and V for every token. Recomputing them is wasteful.

KV Cache — The Problem It Solves

Without caching: $O(n^2)$ work per new token. With caching: $O(n)$ append.

Without KV Cache

- At each new token generation step, the model must recompute K and V matrices for ALL previous tokens from scratch.
- For a 1000-token context generating token 1001: recompute $1000 \times K$ and $1000 \times V$.
- For token 1002: recompute $1001 \times K$ and $1001 \times V$.
- This is $O(n^2)$ total work — prohibitively expensive.



KV Cache
fixes this

With KV Cache

- Compute K and V for each token once. Store them in GPU memory (the KV cache). On each new token step, compute K and V only for the new token and append.
- Each step is now $O(n)$ append, not $O(n^2)$ recompute. This is what makes real-time LLM inference feasible.

What the KV cache stores — per layer, per head, per token:

Key tensors K

One K vector per token per attention head per layer. Shape per layer: (n_heads, seq_len, d_k)

Value tensors V

One V vector per token per attention head per layer. Shape per layer: (n_heads, seq_len, d_v)

NOT the Q tensors

Queries are computed fresh at each step for the new token only. Only K and V are cached because they come from previous tokens.

KV Cache Memory Cost — Longer Context = More VRAM

This is why context window is not free. Memory is the binding constraint in production.

$$\text{KV cache size} = 2 \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{bytes_per_element}$$

Model	Layers	KV heads	d_head	4K tokens	128K tokens
Llama 3 8B (GQA)	32	8	128	~512 MB	~16 GB
Llama 3 70B (GQA)	80	8	128	~1.3 GB	~41 GB
GPT-3 175B (MHA)	96	96	128	~18 GB	~576 GB
Llama 3 8B INT8	32	8	128	~256 MB	~8 GB

GQA (Grouped-Query Attention):

Llama 3 uses 8 KV heads for 32 query heads (4:1 ratio) — 4× KV cache reduction vs standard MHA with minimal quality loss. This is why modern LLMs use GQA.

PagedAttention (vLLM):

Manages KV cache as non-contiguous virtual memory pages, eliminating fragmentation. Enables 128K-token models in production without OOM. Deep-dive in M9.

5.4

API Parameters Map to Generation Concepts

Every parameter controls a mechanism you now understand from Topics 1 and 2

Generation Control Parameters

temperature · top_p · top_k · max_tokens · stop

temperature

float 0.0 – 2.0

0.0 for code/factual (deterministic). 0.7 default (balanced). 1.0+ for creative writing. Never set above 1.5 in production.

Topic 1 · sub-topic 1.4

Scales logits before softmax: $P(x_i) = \exp(z_i/\tau) / \sum \exp(z_j/\tau)$. $\tau \rightarrow 0$ = greedy. $\tau=1.0$ = raw model distribution. $\tau > 1$ flattens, increases diversity.

top_p (nucleus)

float 0.0 – 1.0

0.9–0.95 is the standard production default. Set close to 1.0 for creative tasks. Do not combine with low temperature (redundant).

Topic 1 · sub-topic 1.6

Sample from the smallest set of tokens whose cumulative probability $\geq p$. At $\text{top_p}=0.9$: restricts to tokens covering 90% of probability mass. Eliminates low-probability tail tokens.

max_tokens

integer 1 – context_limit

Always set explicitly. Omitting lets models generate until EOS, which can be slow and expensive. 512 is a reasonable default for chat.

Topic 2 · sub-topic 2.3

Hard ceiling on output token count. When reached, generation halts immediately. Triggers `finish_reason: 'length'` in the API response. Does not affect input token count.

stop (stop sequences)

string or list

Useful for structured output: `['</answer>', '###', '\n\n']`. Essential for code generation to stop at function boundaries.

Topic 2 · sub-topic 2.3

Caller-defined strings that halt generation the moment they appear in output. The stop string is not returned in the response. Multiple stop sequences can be specified. Triggers `finish_reason: 'stop'`.

Context and Control Parameters

stream · seed · system role · logprobs

stream boolean Surfaces the autoregressive token-by-token generation to the caller in real time. Each chunk contains one or more tokens as they are generated. The model is not faster — generation is identical. Only delivery changes. Always use in user-facing apps. Dramatically improves perceived latency. Covered in Assignment A2: stream=True, log each chunk. Topic 2 · A2
seed integer Sets the random seed for the sampling process. At the same temperature and seed, the model produces the same output every time. Not identical to temperature=0 (greedy) — seed fixes randomness, temperature still controls the distribution shape. Use for reproducible experiments, regression testing, and evaluations. Do not rely on in production — minor backend changes can shift output even with a fixed seed. New in Topic 5
system (role) string The system message is prepended to the context window at position 0. Every user and assistant token attends to it via self-attention at every generation step. Changing one word changes the attention pattern for the entire conversation. Keep concise — it costs tokens on every call. Persona, task framing, format instructions, and safety constraints all belong here. 200–500 tokens is a typical target. Topic 3 · self-attention
logprobs boolean / integer Returns the raw log-probabilities from the softmax output for each generated token. Exposes exactly the distribution studied in Topic 1. logprobs=5 returns the top-5 token probabilities at each position. Used in Assignment A1. Production uses: confidence scoring for RAG, uncertainty detection, hallucination risk signals, reranking generated candidates. Topic 1 · A1

Parameter Quick-Reference Card

Task-to-parameter mapping for the six most common production scenarios

Task → Recommended parameter values (all others at model defaults)

Task	temperature	top_p	max_tokens	stop	stream	notes
Factual Q&A / RAG	0.0 – 0.2	0.9	256 – 512	–	true	Low temp = consistent. stream for UX.
Code generation	0.0 – 0.2	0.95	512 – 2048	<code>['\n\n', '']</code>	false	Stop at code block end.
Creative writing	0.8 – 1.2	0.95	512 – 1024	–	true	Higher temp for variety.
Structured output (JSON)	0.0	1.0	256 – 1024	<code>['}']</code>	false	Greedy + stop at JSON close.
Summarisation	0.3 – 0.5	0.9	128 – 512	–	false	Moderate temp, cap output.
Chatbot / assistant	0.7	0.9	512	–	true	Stream always. History = cost.

5.5

Latency, Throughput, and the Cost Tradeoff Triangle

Three quantities that cannot all be simultaneously optimised

Two Latency Components — TTFT and TPOT

Understanding both is essential for production UX and cost estimation

TTFT — Time to First Token

The time between sending the API request and receiving the first output token.

Dominated by: Prefill phase

The model processes the entire input sequence in one parallel forward pass (training-style). All prompt tokens are processed simultaneously.

Longer prompts = longer TTFT.

Retrieval-augmented prompts with 2K+ context tokens have measurably higher TTFT.

Improvement: Flash Attention reduces memory bandwidth (M9). Prompt caching skips re-processing of cached prefixes.

TPOT — Time Per Output Token

The time required to generate each successive output token after the first.

Dominated by: Autoregressive decoding

One forward pass per token — inherently sequential (Topic 2). Cannot be parallelised within a single response.

A 500-token response at 40 tok/s takes 12.5 seconds total.

GPT-4o: ~40-80 tok/s. Llama 3 8B on A100: ~80-120 tok/s.

Improvement: Speculative decoding drafts multiple tokens per pass then verifies (M9). Quantisation increases throughput (M9).

Why stream=True matters for UX: the user sees the first token within TTFT (~500ms) rather than waiting TTFT + all TPOT (~13s). Total time is identical.

The Production Tradeoff Triangle

Latency · Throughput · Cost — pick two. Every production decision is on this surface.

LOW LATENCY

Fast response per user

Tradeoff:

Requires dedicated GPU capacity. One request processed at a time. Cost per query is high.

Example:

Interactive chatbot, voice assistant, code autocomplete

HIGH THROUGHPUT

Many requests per second

Tradeoff:

Continuous batching aggregates requests. Individual latency increases as queue grows. GPU utilisation is high.

Example:

Batch document processing, nightly summarisation pipelines

LOW COST

Minimum \$ per query

Tradeoff:

Smaller models, quantisation, batch API (50% discount). Quality and latency may degrade. Cache system prompts.

Example:

High-volume classification, embedding generation, async pipelines

No system simultaneously achieves low latency + high throughput + low cost. The right point on the triangle depends on your application. Always make this choice explicitly.

Five Cost Reduction Strategies — Applied Today

These apply to every API call you write from M4 onwards

1	Prompt Caching Up to 90% on cached tokens	Anthropic and OpenAI cache frequently-repeated prompt prefixes (system prompts, RAG preambles). The cached portion is not re-processed on subsequent calls. The model sees the same output as if it ran the full prompt.	Keep your system prompt identical across calls. Put stable content first. Variable content (user query) last.
2	Model Routing 3–10× cost reduction	Route simple queries (classification, short Q&A) to smaller cheaper models (GPT-4o-mini at \$0.15/1M input). Route complex multi-step reasoning to frontier models (GPT-4o at \$2.50/1M).	Build a lightweight classifier that scores query complexity. Route based on score threshold. Monitor quality per route.
3	Context Compression 20–60% on history tokens	Summarise conversation history before injecting it into the context window. Replace 2000 tokens of raw history with a 300-token summary. Quality impact is low for task-focused dialogues.	Implement a rolling summariser that activates when history > threshold. Store full history in a database; inject summary only.
4	Output Length Control 10–40% on output tokens	Set max_tokens to the minimum needed for the task. Use structured output formats (JSON, markdown with schema) to avoid verbose prose. Output tokens cost 3–5× more per token than input.	Profile average output length per task type. Set max_tokens = avg + 20% buffer. Use response_format=json_object for structured tasks.
5	Batch API 50% flat discount (OpenAI)	OpenAI Batch API processes requests asynchronously (within 24h) at 50% cost. Suitable for non-latency-critical pipelines: nightly summarisation, embedding generation, bulk evaluation runs.	Identify which pipeline steps are latency-insensitive. Move those to Batch API. Combine with prompt caching for maximum reduction.

The API Call Decision Framework

Ask these six questions before every model call you write from M4 onward

Before writing any model call: work through this checklist. It prevents common production mistakes.

- | | | |
|---|--|--|
| 1 | What is the task?
<i>Factual / code / creative / classification / summarisation. This determines temperature and top_p.</i> | Set temperature: 0.0–0.2 (factual), 0.7 (balanced), 0.9+ (creative). Never guess. |
| 2 | What is the context budget?
<i>Count: system prompt + history + retrieved context + user query. Will this fit in the window?</i> | Calculate token estimate before deployment. Add a safety margin. Plan context compression if needed. |
| 3 | Is streaming needed?
<i>Will a user wait for the full response? Or is this a background pipeline call?</i> | stream=True for any user-facing interface. stream=False for pipeline steps where you need the full output. |
| 4 | What is the output length range?
<i>What is the minimum and maximum useful response length for this task?</i> | Set max_tokens = max useful length + 10% buffer. Use stop sequences for structured output tasks. |
| 5 | Is cost-sensitivity high?
<i>Is this a high-volume pipeline? Does the cost compound across thousands of daily calls?</i> | Consider model routing, prompt caching, or batch API. Profile token counts before deploying. |
| 6 | Is reproducibility required?
<i>Do you need identical outputs for testing, evaluation, or audit purposes?</i> | Set seed for reproducible runs. Set temperature=0 for fully deterministic outputs. |

Four Misconceptions — Cleared

Common errors that persist even after the Topics 1–4 foundation

M1

MYTH: "The context window is a memory that stores information across sessions"

CORRECTION: The context window is cleared completely between API calls. There is no persistent memory. Multi-session memory requires explicit external storage — a vector database, a database, or a file — and injection into the context on each new call.

M2

MYTH: "A longer context window always gives better results"

CORRECTION: More context increases cost, TTFT, and KV cache memory linearly. Beyond a certain length, self-attention can lose focus on key information buried in the middle of a long context ('lost in the middle' phenomenon). Relevant, concise context outperforms long irrelevant context.

M3

MYTH: "Setting temperature=0 is the same as setting a fixed seed"

CORRECTION: temperature=0 applies greedy decoding — always selecting the highest-probability token. A fixed seed at temperature=0.7 introduces reproducible randomness — the same random draw every time but still stochastic. They serve different purposes.

M4

MYTH: "The KV cache makes generation faster by pre-computing the answer"

CORRECTION: The KV cache stores K and V tensors from previous tokens so they are not recomputed. It does not pre-compute anything about future tokens. Generation is still fully sequential — one forward pass per output token. The cache reduces per-step work from $O(n^2)$ to $O(n)$.

References & Further Reading

Papers, documentation, and tools for Topic 5

Papers & Technical Reports

- [1] **Kwon et al. (2023). Efficient Memory Management for LLM Serving with PagedAttention. SOSP.**
Foundational vLLM paper — KV cache as paged virtual memory
- [2] **Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention. NeurIPS.**
IO-aware attention — reduces TTFT for long contexts
- [3] **Leviathan et al. (2023). Fast Inference from Transformers via Speculative Decoding. ICML.**
Drafts multiple tokens per step — reduces TPOT
- [4] **Liu et al. (2023). Lost in the Middle: How LLMs Use Long Contexts. TACL.**
Empirical evidence for attention degradation in very long contexts
- [5] **Peng et al. (2023). YaRN: Efficient Context Window Extension of LLMs.**
RoPE frequency scaling — enables 128K+ contexts post-hoc

Documentation & Tools

- | | |
|------|--|
| Docs | OpenAI API Reference — full parameter documentation
platform.openai.com/docs/api-reference |
| Docs | Anthropic API Reference — prompt caching guide
docs.anthropic.com/en/docs/build-with-claude/prompt-caching |
| Tool | tiktoken — OpenAI tokenizer for token counting
github.com/openai/tiktoken |
| Tool | vLLM — PagedAttention open-source inference engine
github.com/vllm-project/vllm |
| Tool | LangSmith — LLM observability and cost tracking
smith.langchain.com |
| Blog | Lilian Weng — Extrinsic Hallucinations in LLMs
lilianweng.github.io/posts/2024-02-05-human-data-quality |

Topic 5 Summary — LLM APIs Recap

Six essential takeaways

1

Context window = the Transformer's input sequence

Fixed token count covering input + output. When full, oldest tokens are dropped — no gradual fade. The limit is a positional encoding design choice, not just a memory constraint.

3

KV cache trades GPU memory for inference speed

Stores K and V tensors from previous tokens to avoid $O(n^2)$ recomputation. Reduces per-step cost to $O(n)$. Longer context = more VRAM. GQA (Llama 3) reduces KV cache 4× vs standard MHA.

5

TTFT is parallel; TPOT is sequential

Prefill (input processing) is parallel — fast. Decoding (output generation) is one forward pass per token — inherently sequential. `stream=True` shows the user tokens as they arrive, masking total wait time.

2

Tokens are currency — input and output billed separately

Output tokens cost 3–5× more per token. System prompt, history, and RAG context are the largest input cost drivers. A typical RAG query runs ~\$0.01. At 10K queries/day that is \$100/day.

4

Every API parameter maps to a mechanism you know

temperature → softmax scaling (T1). top_p → nucleus sampling (T1). max_tokens / stop → stopping conditions (T2). stream → autoregressive loop surfaced (T2). system → position 0 of context (T3).

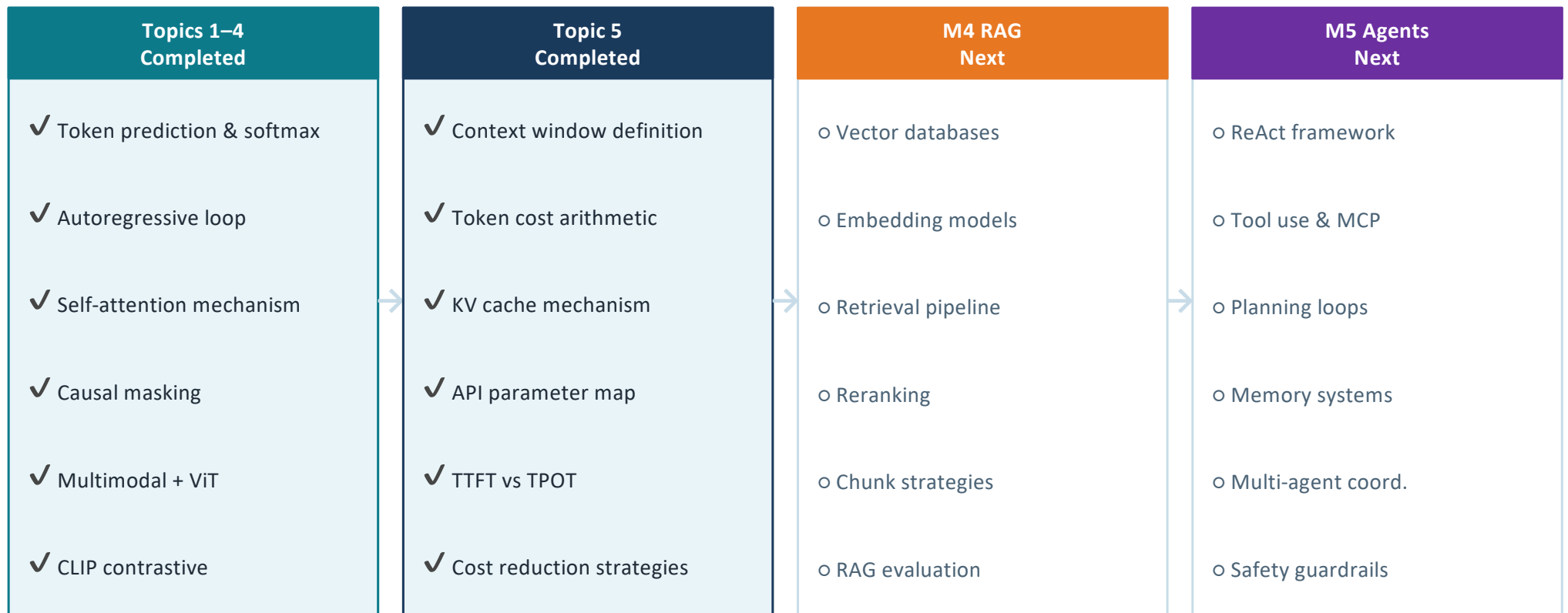
6

Pick your point on the latency–throughput–cost triangle

These three cannot all be simultaneously optimised. Use prompt caching, model routing, context compression, output control, and batch API to reduce cost. Always make the tradeoff explicitly.

Consolidation + APIs — What Carries Forward

The full M1–M3 foundation. Every module from here builds on this.



Assignments A1–A5 remain open until mentor session · Context window + KV cache now fully explained · Apply the decision framework from slide 21 to every M4+ model call

Topic 5 Complete — Consolidation Lecture Done

LLM APIs Recap

Full M1–M3 foundation established. What comes next:

Mentor Sessions

A1–A5 Group Discussion (10 mentors, groups of 5–8)

Submit all five notebooks before the session. Mentors will run group discussions around the worked code. Bring questions.

M4 — RAG

Retrieval-Augmented Generation

Vector embeddings, FAISS / Pinecone, retrieval pipelines, reranking, faithfulness evaluation. The context window and KV cache concepts from Topic 5 are directly relevant to every RAG cost decision.

M5 — Agents

Agentic AI Foundations

ReAct reasoning loops, tool use, MCP (Model Context Protocol), planning, multi-agent coordination. The API parameters from Topic 5 govern every individual model call inside an agent.

Assignments A1–A5 due before mentor session · All five topic decks on the course portal · Context + cost decisions apply from M4 Day 1